Problem Set 1 Solutions

Problem 1

a (**1 point**) Value-based paradigm. Only the traversability metric is parameterized and learned from data. All other parts of the control algorithm leverage a simple model. The metric merely guides policy *evaluation*.

b (**1 point**) Model-based paradigm. The algorithm is only updating the unknown parameters of a neural network model.

c (**1 point**) Combination of value-based and policy-based. The Q-function is a scalar function that provides a measure of performance. In this case, the policy *parameters* are updated using the learned Q-function. This is an actor-critic approach.

d (**1 point**) Model-based paradigm. The algorithm is only learning the local linear model.

Problem 2

a (**3 points**) The node sequence is AGHJB (12).

b (**1 point**) The node sequence is AGHIB (15) or AGHJB (12).

c (**2 points**) Dynamic Programming solves a series of tail problems, starting from the goal and working backwards to the initial state. Dijkstra's algorithm is a "best-first" approach, where the shortest path is found by choosing the closest nodes and working forward through the graph. Dijkstra's algorithm is specialization of Dynamic Programming for the case of positive edge weights and for the situation where each node is only visited once.

Problem 3

a (**1 point**) You need to store 20×25 values in general for a 20-step time horizon for a finite-horizon DP problem.

b (**1 point**) Because VI searches for a fixed-point, only 25 values are needed.

c (**2 points**)

$$T = \begin{bmatrix} 0.0 & 0.1 & 0.2 & 0.7 \\ 0.6 & 0.0 & 0.4 & 0.0 \\ 0.1 & 0.4 & 0.0 & 0.5 \\ 0.8 & 0.0 & 0.2 & 0.0 \end{bmatrix} \tag{1}$$

d (**2 points**) See Matlab solution file.

e (**4 points**) See Matlab solution file.

Problem 4

a (**2 points**)

For this part, we redefine our cost function in MATLAB as

```
[Q,R]=get_QR;

C=(Q(1,1)*X(1,:).^2 + Q(2,2)*X(2,:).^2)+R*u.^2;
```

where $R = 10$, $Q(1,1) = 1$ and $Q(2,2) = 1$. The results of value iteration using this cost function are shown in figure 1.

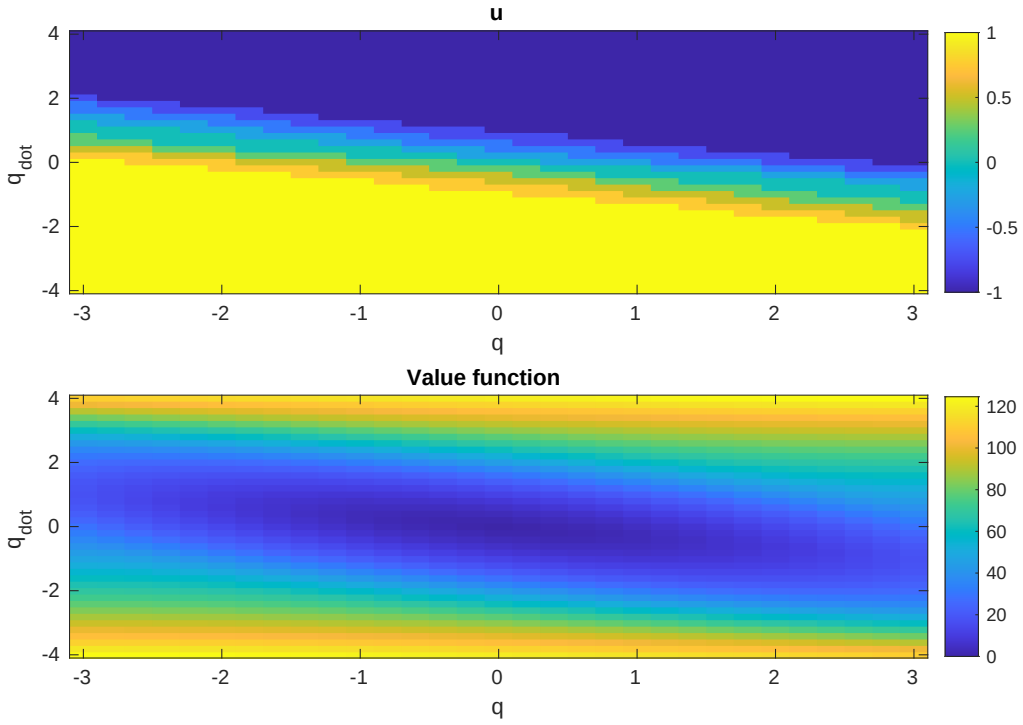

Figure 1: LQR Using Value Iteration

We can then also plot the control and state space trajectories from the initial condition $x_0 = [-1, 1.2]$ as shown in figures 2 and 3.

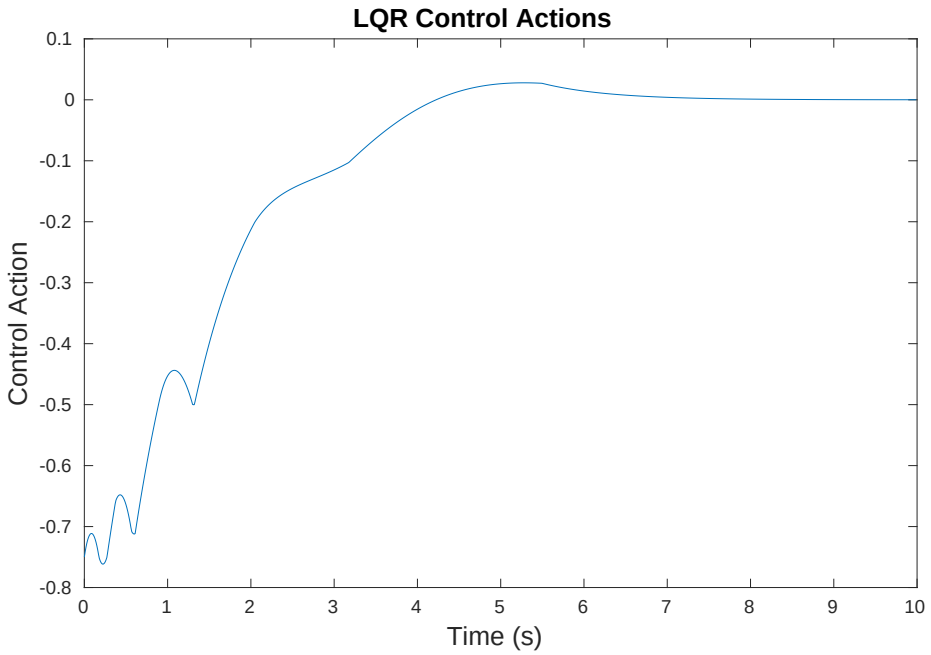

Figure 2: Control Trajectory from Value Iteration LQR

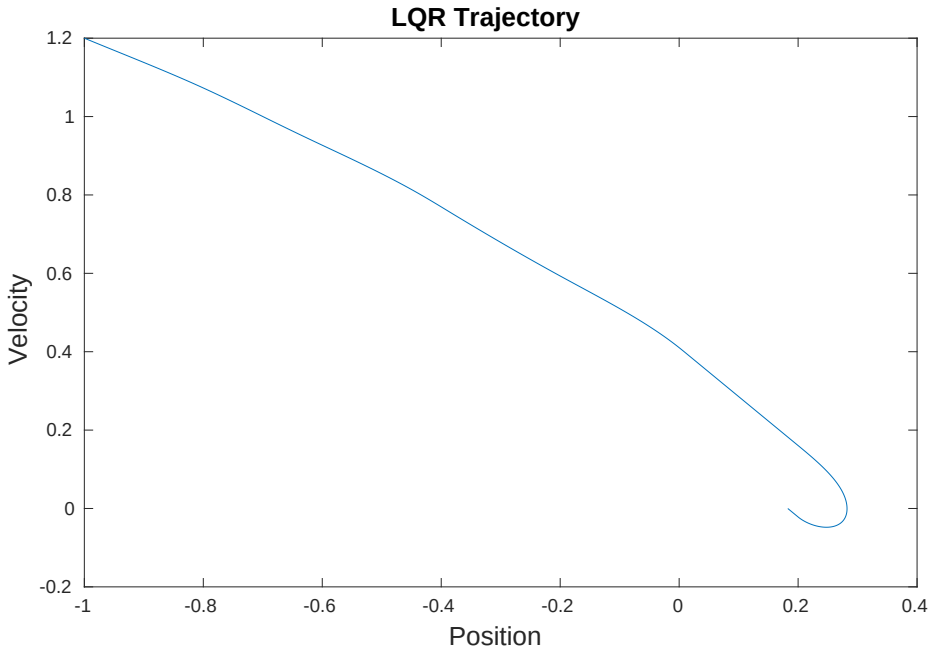

Figure 3: State Space Trajectory from Value Iteration LQR

Using the MATLAB ”lqr” function, we can also find and simulate the LQR Controller for the same initial conditions. The results are shown in figures 4 and 5.

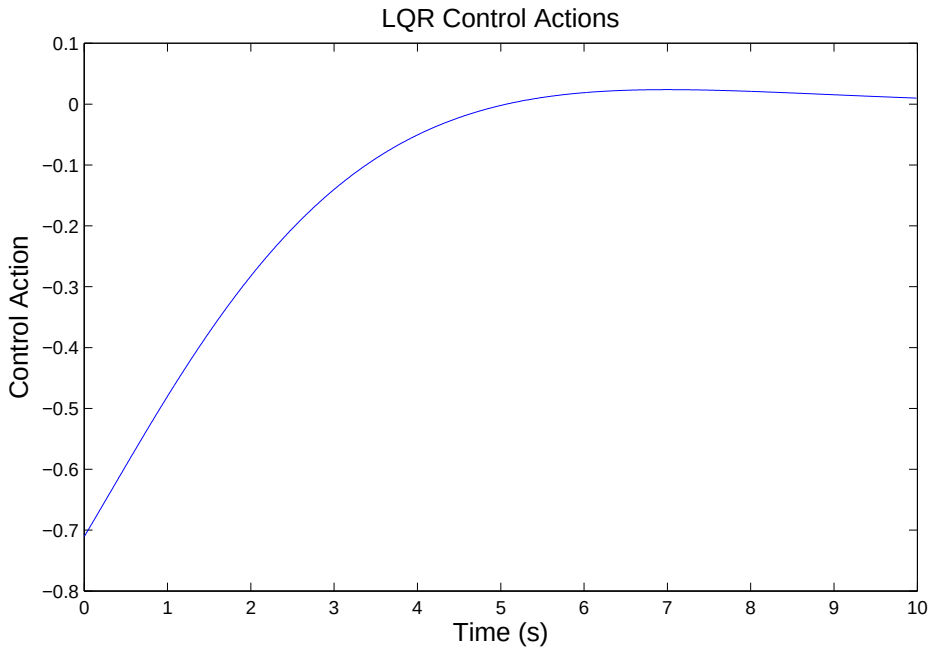

Figure 4: Control Trajectory Using MATLAB LQR

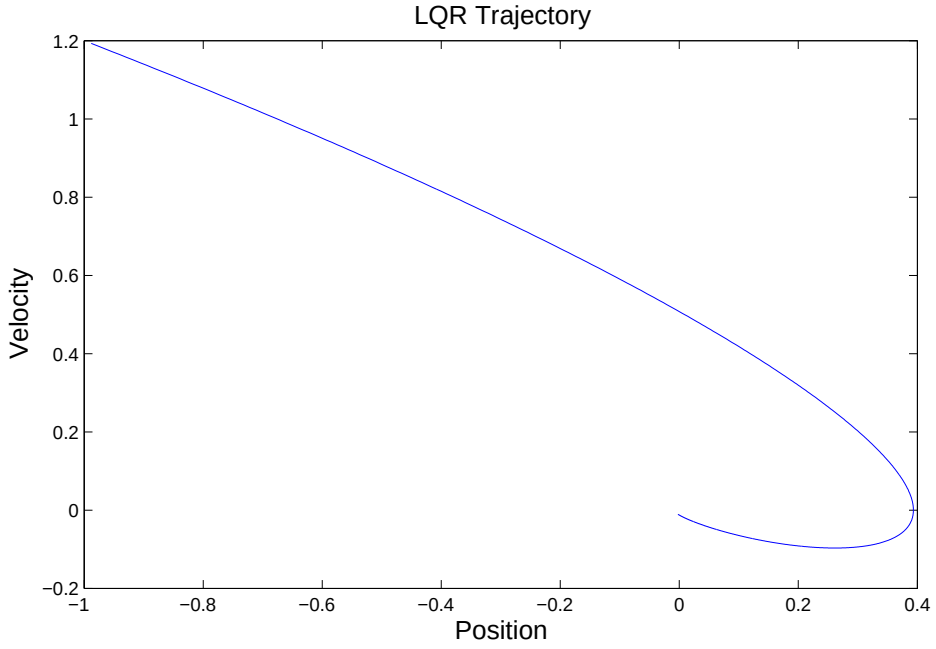

Figure 5: State Space Trajectory Using MATLAB LQR

b (**2 points**)

By examining the plots, one can observe that the value iteration version of LQR differs from the MATLAB version in two major ways. First of all the value iteration version produces a control trajectory which is not as smooth, and secondly, the value iteration does not converge all the way to the goal.

There are a number of reasons why the value iteration version of LQR does not perform as well as the MATLAB

version. The first source of error is discretization. One can imagine that as the mesh grid resolution improves, the policy produced by value iteration will better approximate the results produced by analytical LQR, which is derived for a system continuous in both time and state. Another source of error is the discount factor, γ, which makes J converge faster, but slightly changes the cost function on each time step. For the LQR approach, this is equivalent to adding an exponential decay to the Q and R matrix values. Therefore, over many iterations, the cost function deviates significantly from the MATLAB LQR cost. This explains why the brick does not reach the goal in the value iteration approach. Essentially, after many iterations, the Q matrix is too small to drive the brick to the goal.

Last of all, in value iteration approach, we use volumetric interpolation to compute cost and control between grid points. This interpolation works for the minimum time problem, since the cost is linear, but the quadratic cost of LQR introduces error due to its nonlinearity. The fact that you must interpolate between grid points also explains why the control trajectory is not as smooth.

c (**3 points**) To modify the value iteration code to find the optimal policy for the pendulum, we redefine the dynamics function as follows:

```
function xdot = dynamics(x,u)
 m = 1; % kg
    l = .5; % m
    b = 0.1; % kg m^2 /s
    lc = .5; % m
    I = .25; %m*l^2; % kg*m^2
    g = 9.8;
xdot = [x(2,:); (u - m*g*l*sin(x(1,:)) - b*x(2,:))/I];
end
```

We then change the mesh points as well as the action discretization.

```
% define the mesh points as a row vector
q_bins = linspace(0,2*pi,25);
qdot_bins = linspace(-10,10,25);
% The discrete actions, defined as a row vector
a = linspace(-2,2,5);
```

The results using the minimum time cost function can be found if figure 6.

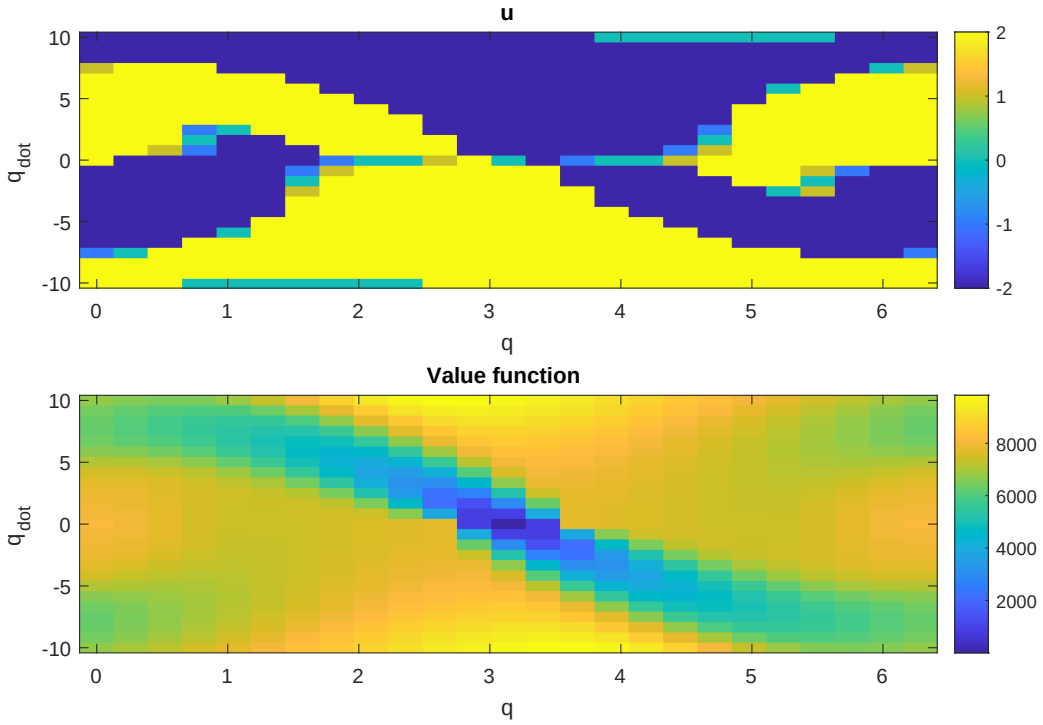

Figure 6: Pendulum Value Iteration

Problem 5

a **(2 points)**

```
%==========================================================================
%  final cost gradients
%==========================================================================
function [gx, gu, gxx, gux, guu] = final_cost_gradients(x,u,xd,Qf)
gx = Qf*(x-xd);
gu = 0;
gxx = Qf;
gux = [0 0 0 0];
guu = 0;
end

%==========================================================================
%  running cost gradients
%==========================================================================
function [gx, gu, gxx, gux, guu] = cost_gradients(x,u,xd,Q,R)
gx = Q*x;
gu = R*u;
gxx = Q;
gux = [0 0 0 0];
guu = R;
end
```

b **(3 points)**

Now we are going to derive the Q-terms for

$$V_k(x_k, u_k) = \min_u Q_k(x_k, u_k)$$

where $Q_k = g(x_k, u_k) + V_{k+1}(x_k, u_k)$
We take second order expansion for cost function

$$g_k = g(*) + g_x^T \delta x_k + g_u^T \delta u_k + \frac{1}{2}\delta x_k^T g_{xx} \delta x_k + \frac{1}{2}\delta u_k^T g_{uu} \delta u_k + \delta u_k^T g_{ux} \delta x_k$$

And for value function

$$\begin{aligned} V_{k+1} &= V_{k+1}(*) + V_x^T \delta x_{k+1} + \frac{1}{2}\delta x_{k+1}^T V_{xx} \delta x_{k+1} \\ &= V_{k+1}(*) + V_x^T (A\delta x_k + B\delta u_k) + \frac{1}{2}(A\delta x_k + B\delta u_k)^T V_{xx}(A\delta x_k + B\delta u_k) \\ &= V_{k+1}(*) + V_x^T A\delta x_k + V_X^T B\delta u_k + \frac{1}{2}\delta u_k^T B^T V_{xx} A\delta x_k + \frac{1}{2}\delta x_k^T A^T V_{xx} A\delta x_k \\ &+ \frac{1}{2}\delta u_k^T B^T V_{xx} B\delta u_k + \frac{1}{2}\delta x_k^T A^T V_{xx} B\delta u_k \end{aligned}$$

Similar with the second order expansion of Q-function

$$Q_k = Q(*) + \begin{bmatrix} Q_x \\ Q_u \end{bmatrix}^T \begin{bmatrix} \delta x_k \\ \delta u_k \end{bmatrix} + \frac{1}{2}\begin{bmatrix} \delta x_k \\ \delta u_k \end{bmatrix}^T \begin{bmatrix} Q_{xx} & Q_{ux}^T \\ Q_{ux} & Q_u u \end{bmatrix} \begin{bmatrix} \delta x_k \\ \delta u_k \end{bmatrix}$$

Then we can write the Q-terms with g_k and V_{k+1}

$$\begin{aligned}
Q_k &= g_k + V_{k+1} \\
&= g(*) + V_{k+1}(*) + (g_x^T + V_x^T A)\delta x_k + (g_u^T + V_X^T B)\delta u_k + \frac{1}{2}\delta x_k^T (g_{xx} + A^T V_{xx} A)\delta x_k \\
&+ \frac{1}{2}\delta u_k^T (g_{uu} + B^T V_{xx} B)\delta u_k + \frac{1}{2}\delta x_k^T (g_{ux}^T + A^T V_{xx} B)\delta u_k + \frac{1}{2} u_k^T (g_{ux} + B^T V_{xx} A)\delta x_k
\end{aligned}$$

Now the Q-terms are straightforward

$$\begin{aligned}
Q_x &= g_x^T + V_x^T A \\
Q_u &= g_u^T + V_x^T B \\
Q_{xx} &= g_{xx} + A^T V_{xx} A \\
Q_{uu} &= g_{uu} + B^T V_{xx} B \\
Q_{ux} &= g_{ux} + B^T V_{xx} A
\end{aligned}$$

(2 points)

```
%====================================================================
%  get Q-terms
%====================================================================
function [Qx, Qu, Qxx, Qux, Quu, Quubar, Quxbar]= Q_terms (gx, gu, gxx,
    gux,guu, fx, fu, Vx, Vxx)
   Qx= gx + fx'*Vx;
   Qu = gu + fu'*Vx;
   Qxx = gxx+ fx'*Vxx*fx;
   Qux = gux + fu'*Vxx*fx;
   Quu = guu + fu'*Vxx*fu;
   rho=0.00;
   Quxbar = gux + fu'*(Vxx+rho*eye(length(Vxx)))*fx;
   Quubar = guu + fu'*(Vxx+rho*eye(length(Vxx)))*fu;
end
```

c **(2 point)**

```
   %====================================================================
   %  get gains
   %====================================================================
   function [K, v]= gains(Qx,Qu,Qxx,Qux,Quu)
      K = -inv(Quu)*Qux;
      v = -inv(Quu)*Qu;
   end
```

d **(3 points)**

Plugging back the optimal control law into the Bellman Equation

$$\begin{aligned}
V_k(*) + V_x^T \delta x + \frac{1}{2}\delta x^T V_{xx} \delta x &= Q_k(*) + Q_x \delta x_k + Q_u \delta u_k + \frac{1}{2}\delta x_k^T Q_{xx} \delta x + \frac{1}{2}\delta u_k^T Q_{uu} u_k \\
&+ \frac{1}{2}\delta u_k^T Q_{ux} x_k + \frac{1}{2}\delta x_k^T Q_{ux}^T u_k \\
&= Q_k(*) + Q_x \delta x_k + Q_u(K\delta x_k + k) + \frac{1}{2}\delta x_k^T Q_{xx} \delta x \\
&+ \frac{1}{2}(K\delta x_k + k)^T Q_{uu}(K\delta x_k + k) + \frac{1}{2}(K\delta x_k + k)^T Q_{ux} \delta x_k \\
&+ \frac{1}{2}\delta x_k^T Q_{ux}^T (K\delta x_k + k) \\
&= (Q_x + Q_u K + k^T Q_{uu} K)\delta x_k \\
&+ \frac{1}{2}\delta x_k^T (Q_{xx} + K^T Q_{uu} K + K^T Q_{ux} + Q_{ux}^T K)\delta x_k + Q
\end{aligned}$$

We thus obtain the V-terms as follows:

$$\begin{aligned}
V_x &= (Q_x + Q_u K + k^T Q_{uu} K)^T = Q_x + K^T Q_u + K^T Q_{uu} k + k^T Q_{ux} \\
V_{xx} &= Q_{xx} + K^T Q_{uu} K + K^T Q_{ux} + Q_{ux}^T K
\end{aligned}$$

After plugging back control gains K and k, the V-terms becomes

$$\begin{aligned}
V_x &= Q_x + K^T Q_u \\
V_{xx} &= Q_{xx} + K^T Q_{ux}
\end{aligned}$$

(2 points)

```
%=====================================================================
%  get V-terms
%=====================================================================
function [Vx, Vxx] = Vterms (Qx,Qu,Qxx,Qux,Quu,K,k)
   Vxx = Qxx + K'*Qux;
   Vx = Qx + K'*Qu;
end
```

e **(1 point)**

```
u = utraj0(:,i)+ alpha*ktraj(:,i) + Ktraj(:,:,i)*(x-xtraj0(:,i));%compute
    input
```

f **(2 points)** Parameters that lead to successful swing-up are acceptable. One example:

```
Q = 0*eye(4,4);
Qf= 1000*eye(4,4);
R = 0.01;
```

(1 points) See MATLAB solution file.

(2 points) figures.

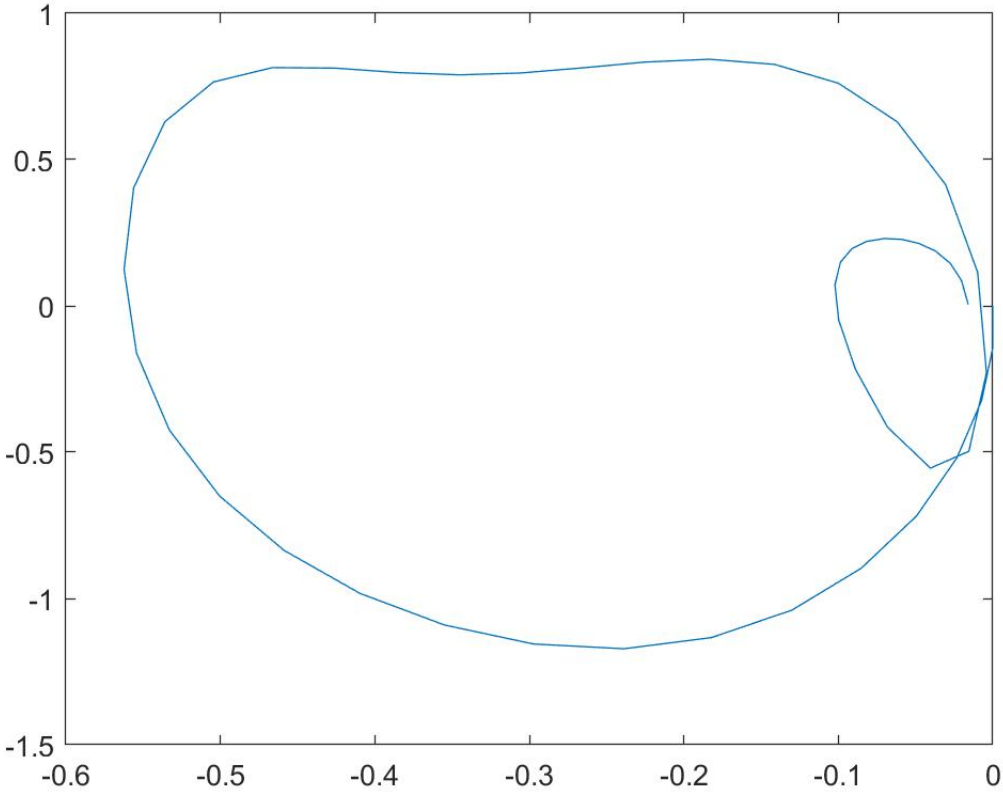

Figure 7: Result trajectory

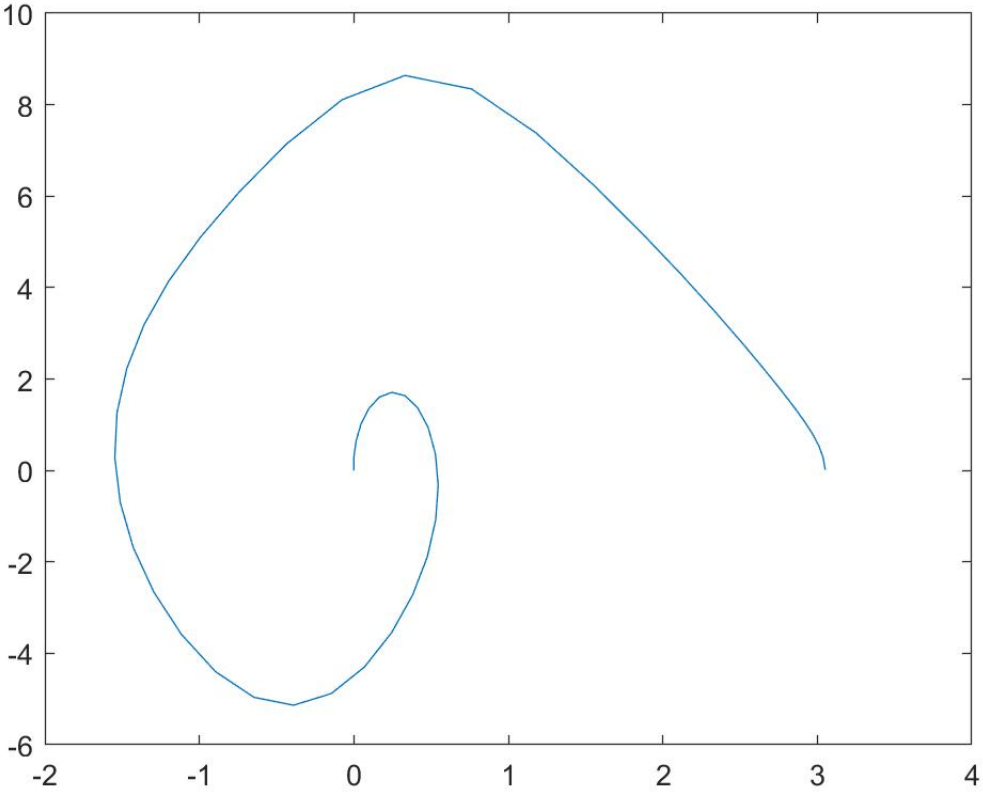

Figure 8: Result trajectory